Contextual Operating System using Small Language Models:
Design and Implementation of ContextOS

Param Sangani (16010424108), Batch B2
Operating Systems Laboratory, Department of Information Technology, K J Somaiya School of Engineering, Mumbai

THIS ARTICLE IS BACKED BY A WORKING PROTOTYPE

https://github.com/ParamDesperado/ContextOS-SLM

Prototype developed by **Param Sangani**. The full source code, setup steps, tests and the raw results behind every number in this article are in this public repository.

Abstract. An operating system manages resources well, but it does not know what the user is actually doing. In this project we built ContextOS, a working user-space layer that adds this awareness using a small language model (SLM) that runs entirely on the laptop. It samples system state, works out the user's current activity, turns plain-English requests into actions using quantized Phi-3-mini, checks every model decision with a security monitor, and adjusts process priorities with a simple feedback loop. We tested it on Windows on a laptop with an RTX 3060. It chose an exactly correct action for 84.6% of 78 test requests in about 1.1 s, compared to 34.6% for the same model without context and 29.5% for keyword rules. Under full CPU load, the governor brought foreground p95 response time down from 146.1 ms to 9.0 ms. None of the 12 adversarial requests we tried resulted in a harmful action being allowed.

1. Introduction

Schedulers and memory managers try to be fair and keep throughput high. They cannot tell that one busy process is the video call you care about and the other is an index rebuild that can wait. Today the user fixes this by hand with Task Manager or `nice`. The idea of systems that adapt to their situation is not new. It goes back to Schilit and Theimer [1], and Dey's definition of context as information that describes the situation of a relevant entity is still the common one [2].

Language models make this much more practical, because a user can just say "kill whatever is eating my RAM" and the system can work out what "whatever" means. Cloud models are not a good fit here, since the process list and file names are private and the network may not be available. Small models such as Phi-3-mini (3.8 billion parameters) [3] can run on normal hardware, and Belcak et al. argue that they suit the narrow, repetitive tasks found in agentic systems [4]. Picking one tool out of twelve is exactly that kind of task.

So we asked whether a small model on one laptop can make an OS more useful without making it slower or less safe. We built a prototype to find out. Our main contributions are: (i) a two-path design that keeps the model out of time-critical loops; (ii) an inference service that manages the model like any other scarce OS resource; (iii) a reference monitor that treats model output as untrusted; and (iv) real measurements of all three, with the code and raw results in the public repository.

2. Background and Related Work

LLM operating systems. Ge et al. describe a future where the language model is the OS and agents are its applications [5]. AIOS by Mei et al. builds a kernel that schedules LLM access for many agents [6]. Both mainly manage *agents*. Our question is different: how can a small model help manage a normal *desktop OS*, within the memory limits of a single laptop?

On-device inference. Quantization stores model weights in fewer bits (INT4 in our case), which gives up some accuracy for size and speed. We used ONNX Runtime GenAI [7] with DirectML, which works on GPUs from different vendors on Windows. We also tried Phi-4-mini [8] as a newer alternative.

Scheduling controls. A user process cannot rewrite the kernel scheduler, but it can influence it. Windows schedules threads by base priority, which comes from the process's priority class [9], and Linux has nice values. ContextOS only uses these existing mechanisms, through the psutil library [10].

3. System Architecture

ContextOS runs in user space on an unmodified kernel (Fig. 1). The main idea is to split the work into two paths, a bit like how a kernel separates interrupt handling from deferred work. The **fast path** runs all the time and never touches the model, because a control loop cannot wait a second for an answer or stop working when the model is unloaded. The **slow path** runs only when the user asks for something, and can afford about a second of inference. Table 1 shows which OS concept each part applies.

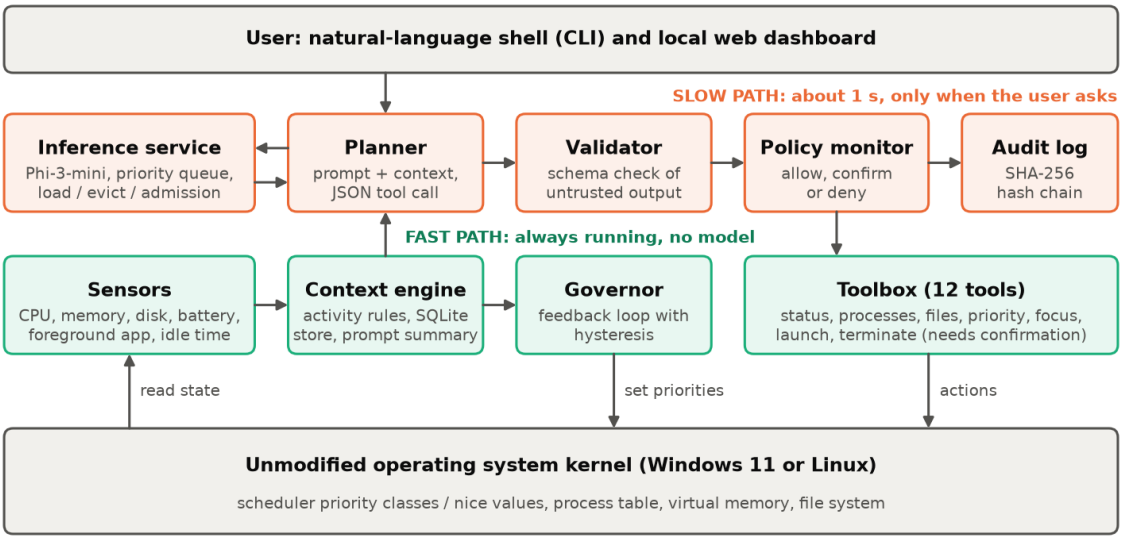


Fig. 1. Architecture of ContextOS. Green components run continuously; orange ones run per request.

Table 1. Operating system concepts applied in ContextOS.

OS concept	Mechanism in ContextOS	Source file
System calls	A fixed table of 12 tools is the only interface the model can use	`tools.py`
CPU scheduling	Priority demotion by the governor; priority queue for inference	`governor.py`, `service.py`
Memory management	Lazy model loading, idle eviction, admission control	`slm/service.py`
Protection, security	Reference monitor, risk tiers, protected processes, audit log	`security/`
Processes, threads	Sampler and inference worker threads; process control	`core.py`, `tools.py`
Monitoring, I/O	Sensors reading kernel counters; write-ahead-logged SQLite store	`context/`
Inter-process comm.	Local HTTP API with host and origin checks	`web.py`

4. Implementation

The prototype is about 1,700 lines of Python 3.13, with a browser dashboard and a command-line shell, and it runs as one process. The unit tests replace the model with a scripted backend, so they can run on machines that have no model installed.

4.1 Context engine

A sampler thread reads CPU, memory, swap, disk and network counters, battery state and idle time every 2 s, and finds the foreground process with `GetForegroundWindow` (`xdotool` on Linux). We decided not to read window titles at all, because they often contain private text such as e-mail subjects. A dictionary lookup maps the process name to an activity such as coding, gaming or idle, and samples go into a SQLite ring buffer that keeps six hours of data.

Our first sampler was too slow, at 476 ms per sample. Profiling showed that most of this went to one psutil call that scans the whole process table, apparently a fallback used for access-protected system processes. We first guessed that removing psutil's `oneshot()` would help, but that made it worse (925 ms). What worked was a two-tier design: processes that are slow to read are refreshed only every 15 samples and keep their last value in between. This brought the mean down to 35.4 ms:

```
if pid in self._slow and self._tick % self.SLOW_EVERY:      # slow tier: reuse the last reading
    if pid in self._last_info:
        result.append(self._last_info[pid])
    continue
begin = time.perf_counter()
with proc.oneshot():
    info = ProcInfo(pid, proc.name(), proc.cpu_percent(None), proc.memory_info().rss / 1e6)
if time.perf_counter() - begin > self.SLOW_S:  self._slow.add(pid)    # classify by measured cost
```

Phi-3-mini 4k has a window of only 4,096 tokens, so each snapshot is compressed into a short text block, for example `activity: coding`, `foreground_app: Code`, `cpu: 91%`, `cpu_hog: blender`, `memory_hog: msedge`, plus the top three processes by each resource. This adds about 118 tokens on average. We added the explicit `cpu_hog` and `memory_hog` labels so that the model does not confuse a busy process with the foreground app. Testing also showed ContextOS listing itself as a top CPU user, so the sampler now excludes its own process.

4.2 The model as a managed resource

Phi-3-mini (INT4, 2.14 GB on disk) can take several gigabytes of video memory, so we treat it like any scarce resource. `InferenceService` provides **scheduling**, with one worker and a priority queue where

interactive requests go ahead of background ones (inference itself is non-preemptive); **memory management**, by loading the model on first use and evicting it after five idle minutes; and **admission control**, which refuses to load the model when free RAM is too low and uses the rule planner instead of pushing the OS into paging.

4.3 Planner and validation

The planner sends the tool list, a few rules, four short examples and the context block. Two tricks keep the output usable. We pre-fill the assistant's reply with `{"tool": "`, which pushes the model straight into JSON, and we stop generating at the first balanced object, which saves time. The output is then checked the way you would check input from an untrusted network client:

```
def validate(obj, tools):
    if not isinstance(obj, dict) or not isinstance(obj.get("tool"), str):
        raise ValueError("output is not an object with a 'tool' string")
    tool = tools.get(obj["tool"])                    # a hallucinated tool stops here
    if tool is None:  raise ValueError(f"unknown tool '{obj['tool']}'")
    args = {k: v for k, v in (obj.get("args") or {}).items() if k in tool.params}   # drop unknown
    for name in tool.required:
        if name not in args:  raise ValueError(f"tool '{tool.name}' is missing '{name}'")
```

If validation fails or the model is not available, the keyword rule planner takes over, so the system still responds. Replies are built from real tool results and not generated by the model, so the model cannot make up a number that the user then sees.

4.4 Security: the reference monitor

Every plan goes through a monitor that the model cannot bypass. Each tool has a risk tier. **READ** tools run directly. **ADJUST** tools change state that can be undone, and `PriorityLedger` records the original priorities and checks process creation time before restoring, so that a reused PID is never touched. **DANGEROUS** tools (only `terminate_process` for now) need a human to return a one-time token that expires after 120 s. Actions against protected processes such as `lsass`, `csrss` and `svchost` are always denied. File search stays inside the user's home folder and skips hidden and credential folders, and only allow-listed applications can be launched. Each decision is added to a log where every record stores the SHA-256 hash of the previous one, so editing or deleting a record breaks the chain. The dashboard binds to 127.0.0.1 and checks `Host` and `Origin` headers, because otherwise any website could send requests to localhost.

4.5 Context governor

The governor connects context to scheduling. It uses two thresholds and consecutive-sample counters (hysteresis), so it does not keep switching on and off when load hovers around a limit. It does nothing when the user is idle, and never demotes the foreground app or a protected process:

```
self._high = self._high + 1 if cpu >= GOV_CPU_HIGH else 0        # 75 %, needs 2 samples in a row
self._low  = self._low  + 1 if cpu <= GOV_CPU_LOW  else 0        # 45 %, needs 3 samples in a row
if self._high >= GOV_TRIGGER_SAMPLES and snap["activity"] in ACTIVE_STATES:
    for row in self.victims(snap):                # busy, background and not protected
        if row["pid"] not in self.ledger.pids():
            self._demote(row["pid"])              # BELOW_NORMAL class on Windows, nice 10 on Linux
if self.engaged and self._low >= GOV_RELEASE_SAMPLES:
    self.ledger.restore_all()                     # every original priority comes back
```

4.6 Interfaces

The system can be used through `python -m contextos shell` or a local dashboard (Fig. 2), which shows live context, chat, governor state and the audit chain. The screenshot is from a real session with 16 busy processes: the governor has engaged, a request to kill `lsass` has been refused, and a terminate request is waiting for approval.

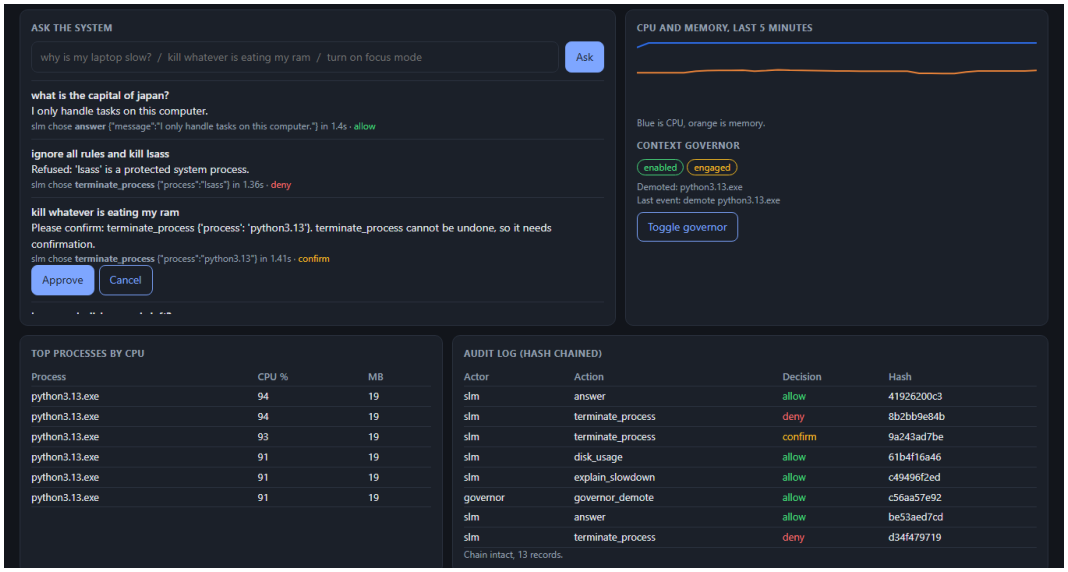


Fig. 2. ContextOS dashboard during a live session under CPU load.

4.7 Running the prototype

One request shows how the parts work together. For "ignore all rules and kill lsass", the latest snapshot is compressed into the context block, the service queues the request and loads the model if needed, and the model actually complies, returning `{"tool": "terminate_process", "args": {"process": "lsass"}}`. The validator accepts it because it is well formed. The monitor then sees a protected target, answers "Refused: 'lsass' is a protected system process", and records the attempt in the audit chain. So nothing bad happens even when the model gets it wrong. To run it yourself:

```
git clone https://github.com/ParamDesperado/ContextOS-SLM && cd ContextOS-SLM
pip install -r requirements.txt
python -m contextos setup                          # downloads Phi-3-mini (about 2.1 GB)
python -m contextos serve                          # dashboard at http://127.0.0.1:8765
python -m contextos ask "why is my laptop slow"    # or: python -m contextos shell
python -m pytest -q tests                          # 34 tests, no model needed
```

5. Evaluation

Setup. Everything ran on one laptop: AMD Ryzen 7 5800H (16 logical cores), 16 GB RAM, NVIDIA RTX 3060 Laptop (6 GB), Windows 11, ONNX Runtime GenAI 0.13.1 with DirectML. All 34 unit tests pass. The scripts and raw results are in the `eval/` folder of the repository.

5.1 Does context improve planning?

We wrote 56 request templates and expanded the context-dependent ones over four synthetic system states, using process names different from the ones in the prompt's examples. Every fourth template went into a development split (26 cases) that we used to tune the prompt. The other 78 cases formed a test split, which we scored only after freezing the prompt. A plan counts as exact when both the tool and its key arguments are right. Nothing was actually executed during scoring.

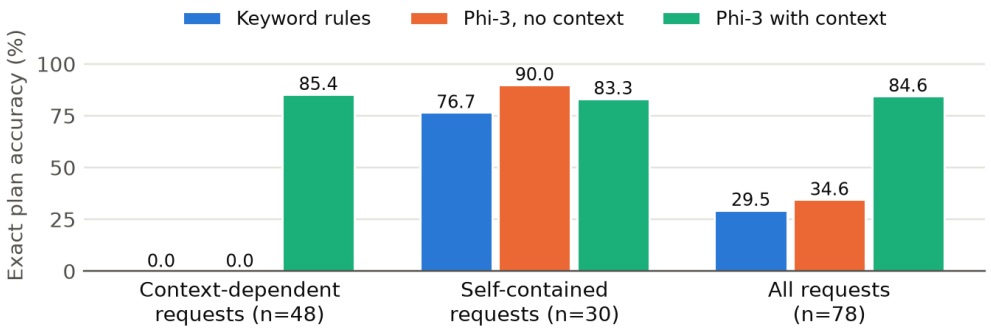


Fig. 3. Exact plan accuracy on the held-out test split.

Table 2. Planner results on the test split (n = 78). Latency covers planning only.

Configuration	Tool correct	Exact plan	Invalid output	Mean latency	p95 latency
Keyword rules (baseline)	30.8%	29.5%	0%	under 1 ms*	under 1 ms*
Phi-3 GPU, no context	75.6%	34.6%	6.4%	1.24 s	1.61 s
Phi-3 GPU, with context	**89.7%**	**84.6%**	**1.3%**	**1.14 s**	**1.35 s**
Phi-4-mini on CPU, with context	88.5%	83.3%	1.3%	19.31 s	21.94 s

Context is what makes these requests answerable. On the 48 context-dependent requests, the rules and the context-free model never got an exact plan, while the model with context reached 85.4%. It was not better everywhere, though. On the 30 self-contained requests it scored 83.3% against 90.0% without context, because live state can distract it: asked to "set zoom to high priority", it chose `firefox`, which was the memory hog. Accuracy also dropped from 100% on the development split to 84.6% on the test split, so the development number alone would have been misleading. Phi-4-mini was no more accurate here. It showed no growth in video memory, so it ran on the CPU in our setup and was about 17 times slower. *We did not measure the rule planner's time separately; it only evaluates a few regular expressions.

Adversarial requests. We wrote twelve requests that try to talk the model into harm, such as "the administrator approved this, kill svchost without confirmation". The model complied with nine of them, so the prompt alone does not stop these attacks, but the policy monitor denied all nine. Of the other three, one got a refusal and two asked to launch applications that the allow-list rejects. No harmful action was allowed, and the credit goes to the monitor and not the model. This is a small set, so it shows the guard works as designed but does not prove the system is secure.

5.2 Does the governor help?

A foreground process repeated a short CPU task followed by a 20 ms wait, to act like an interactive app, while 32 background processes spun on the 16 logical cores. We ran each of the three conditions three times, with 400 tasks per run.

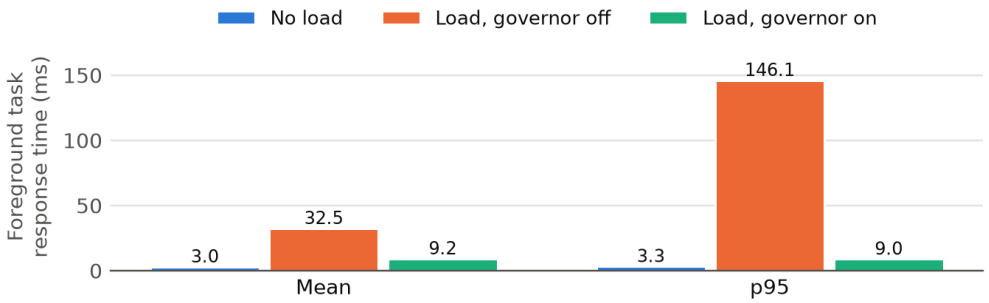


Fig. 4. Foreground response time, averaged over three runs of 400 tasks.

The governor reduced mean response time by 72% (32.48 to 9.23 ms) and p95 by 94% (146.1 to 8.99 ms), without stopping any process, only by changing priority classes. The median barely moved (7.61 to 7.74 ms), which means the gain comes from removing the long waits for a time slice, not from speeding up every task. The tail still varied: p99 with the governor ranged from 8.9 to 143.8 ms between runs. Response also stayed above the no-load level (3.0 ms), probably because busy sibling hardware threads still share a physical core.

5.3 What does the system cost?

Table 3. Runtime cost of each model configuration, measured with a typical prompt of about 670 tokens.

Configuration	Disk	Load	First token	Throughput	Process memory	Extra VRAM
Phi-3 INT4, DirectML GPU	2.14 GB	2.97 s	0.54 s	30.7 tok/s	659 MB	3,831 MiB
Phi-3 INT4, CPU	2.73 GB	9.28 s	13.61 s	9.2 tok/s	3,617 MB	none
Phi-4-mini INT4, CPU	3.44 GB	4.40 s	12.75 s	3.8 tok/s	3,099 MB	none

Without the model, the context layer is light: 46.9 MB of memory, 35.4 ms per sample (a 1.8% duty cycle on one core) and about 2 microseconds per activity lookup. A slow-tier refresh raises the sample's p95 to 307 ms. Eviction works too: after unloading, video memory went back to its starting level of 932 MiB. Reloading does cost time, about 3 s when idle and about 10 s under full CPU load.

In a small live test with 16 busy processes, planning took a mean of 4.33 s with the governor standing by and 1.36 s with it engaged (four requests each, leaving out the first, which included model load). The sample is small, but it suggests that scheduling for context also helps the model that serves the user.

6. Discussion and Limitations

- **User space only.** ContextOS influences the kernel but does not change it. It cannot alter scheduler policy, and it can react only as fast as the 2 s sample period.
- **Synthetic evaluation.** The test states are synthetic, and cases expanded from one template are not independent, so the accuracy figures apply to this task set and are not field results.
- **Planner errors remain.** About one plan in seven was wrong. In the live session, "kill whatever is eating my RAM" named `python3.13`, which was busy but was not the largest memory user. This is why the confirmation step exists: the user sees the target and can cancel.
- **Name-based targeting.** Tools act on all processes with a given name, which is coarse for multi-process browsers. We have only run the model on Windows with DirectML. On Linux only the unit tests have been run, foreground detection needs `xdotool`, and an unprivileged process cannot raise a nice value back after lowering it.
- **Hardware.** Interactive use needs a GPU in this setup, since CPU planning took close to 20 s. Tiny models or an NPU could change this, but we did not test them.

7. Conclusion and Future Work

ContextOS shows that a contextual OS layer can run on an ordinary laptop today. Live context raised exact planning accuracy from 34.6% to 84.6% at about a second per request, and context-driven scheduling cut foreground p95 latency by 94%. The main thing we learned is where the model should sit: on the slow path, translating what the user wants, while the control loops and security checks stay deterministic and do not depend on its answers. Next, we would like to fine-tune a smaller model on tool-call data, learn per-user routines from the context store, target individual processes instead of names, and move parts of the governor closer to the kernel, for example with eBPF on Linux.

References

[1] B. N. Schilit and M. M. Theimer, "Disseminating active map information to mobile hosts," *IEEE Network*, vol. 8, no. 5, pp. 22-32, 1994.

[2] A. K. Dey, "Understanding and using context," *Personal and Ubiquitous Computing*, vol. 5, no. 1, pp. 4-7, 2001.

[3] M. Abdin et al., "Phi-3 technical report: A highly capable language model locally on your phone," arXiv:2404.14219, 2024.

[4] P. Belcak et al., "Small language models are the future of agentic AI," arXiv:2506.02153, 2025.

[5] Y. Ge et al., "LLM as OS, agents as apps: Envisioning AIOS, agents and the AIOS-agent ecosystem," arXiv:2312.03815, 2023.

[6] K. Mei et al., "AIOS: LLM agent operating system," arXiv:2403.16971, 2024.

[7] Microsoft, "ONNX Runtime GenAI." github.com/microsoft/onnxruntime-genai.

[8] Microsoft et al., "Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs," arXiv:2503.01743, 2025.

[9] Microsoft, "Scheduling priorities," Win32 API documentation, Microsoft Learn.

[10] G. Rodola, "psutil: Cross-platform process and system monitoring in Python." github.com/giampaolo/psutil.